

CSE 144 Final Project Report
Transfer Learning Challenge
Group Members: Solara Virak Chum and Samarth Ramesh

1 Introduction

For this project, we worked on the transfer-learning image classification challenge. The goal was to predict one of 100 numeric labels for each test image using a pretty small training set. Since our local training set had 1079 images across 100 classes, there were not enough examples to train a large deep model from scratch reliably. Because of this, we used transfer learning: pretrained ConvNeXt, Swin Transformer, and DINOv2 models extracted image embeddings, and a balanced logistic regression classifier made the final predictions.

Our best setup reached 0.8767 +/- 0.0151 local 4-fold cross-validation accuracy and received an 85.454 public Kaggle leaderboard score.

2 Dataset

- Classes: 100 image classes labeled by folder name, train/0 through train/99.
- Training data: approximately 10 labeled color images per class, with varying image shapes.
- Test data: unlabeled images in test/. The handout describes 1000 images, but the final Kaggle submission generated by the repo contains 1036 rows because Kaggle expected predictions for all 1036 available test files.
- Label mapping: folder 37 maps directly to label 37. The data loader sorts class folders and image filenames numerically to prevent accidental label permutation.
- Preprocessing: each timm model uses its resolved evaluation transform, including model-specific resizing/cropping, tensor conversion, and ImageNet-style normalization.

3 Implementation

The final system was an embedding ensemble followed by balanced logistic regression. We used three pretrained timm encoders: ConvNeXt Base, Swin Base, and a DINOv2 ViT. Each encoder was created with num_classes=0 so that it returned a representation vector instead of logits. Every feature block was L2-normalized, multiplied by a tuned block weight, concatenated, and L2-normalized again before classification.

The three feature extractors were:

- convnext_base.fb_in22k_ft_in1k_384, feature weight 0.25
- swin_base_patch4_window12_384.ms_in22k_ft_in1k, feature weight 0.75
- vit_base_patch14_reg4_dinov2.lvd142m, feature weight 1.5

The submitted blend weights were 1, 0, 0, so the final predictions came from balanced multinomial logistic regression. The code also supports prototype and cosine k-nearest-neighbor probability models, but they did not improve the final submitted setup.

Training and inference details:

- Classifier: sklearn LogisticRegression with `class_weight="balanced"`, `solver="lbfgs"`, `max_iter=5000`, and `C=200`.
 - Validation: stratified 4-fold cross-validation, capped by the smallest class count when needed.
 - Batching: feature extraction uses batch size 16 and two workers in the submitted script.
 - Caching: extracted train/test embeddings are cached in artifacts/features so repeated experiments do not rerun the expensive pretrained encoders.
 - Device choice: the project automatically selects CUDA, then Apple MPS, then CPU.
- We also implemented a ConvNeXt fine-tuning pipeline with AdamW, cosine learning-rate decay, label smoothing, mixup, RandAugment, random erasing, and an optional full-train checkpoint. We kept that path in the repo for comparison, but the frozen-embedding pipeline ended up being the cleaner final submission.

4 Experiments

The main tuning loop evaluated stronger pretrained features and then adjusted block weights and classifier regularization. The progression shows that adding DINOv2 features gave the largest improvement, and weighting the DINOv2 block most heavily produced the best local validation estimate.

Experiment summary:

- Run 1: ConvNeXt embedding quick run
 - Validation accuracy: 0.7692 +/- 0.0087
 - Kaggle public accuracy: not submitted
 - Notes: single-backbone smoke test.
- Run 2: ConvNeXt + Swin embedding ensemble
 - Validation accuracy: 0.7905 +/- 0.0107
 - Kaggle public accuracy: not submitted
 - Notes: two pretrained encoders.
- Run 3: ConvNeXt + Swin tuned ensemble
 - Validation accuracy: 0.8035 +/- 0.0100
 - Kaggle public accuracy: not submitted
 - Notes: tuned two-backbone settings.
- Run 4: ConvNeXt + Swin + DINOv2 ensemble

- Validation accuracy: 0.8656 +/- 0.0122
 - Kaggle public accuracy: not submitted
 - Notes: DINOv2 features improved separation.
-
- Run 5: Block-weighted DINOv2 ensemble
 - Validation accuracy: 0.8767 +/- 0.0151
 - Kaggle public accuracy: 85.454
 - Notes: submitted model; feature weights 0.25, 0.75, 1.5; logistic regression only.
-
- Run 6: ConvNeXt fine-tuning path
 - Validation accuracy: exploratory
 - Kaggle public accuracy: not final
 - Notes: implemented for comparison; embedding pipeline was more reliable for the final submission.

5 Results

- Final local validation: 0.8767 +/- 0.0151 accuracy under 4-fold stratified cross-validation.
- Final Kaggle public leaderboard score: 85.454.
- Final CSV: submissions/SamarthRameshVirakChumKaggleSubmission_1036.csv.
- Submission format check: 1036 rows, columns ID and Label, labels spanning 0 through 99.

The public leaderboard evaluates only a subset of the hidden test set, so it was treated mainly as a format and sanity check rather than as the sole model-selection signal.

6 Discussion

The most important thing we learned from this project is that the representation mattered more than making the classifier complicated. Since the dataset had 100 classes but only a small number of labeled images per class, training a large model directly on the project data would have been risky and likely to overfit. Using pretrained encoders gave the model a much stronger starting point because those networks had already learned general visual patterns from much larger datasets.

In our experiments, the biggest improvement came when we added DINOv2 features to the ConvNeXt and Swin embeddings, which raised local cross-validation accuracy from around 0.8035 in the tuned two-backbone setup to 0.8656 with the three-backbone setup. After weighting the feature blocks, the best local validation score increased again to 0.8767 +/- 0.0151, and this was the version that produced our accepted Kaggle score of 85.454. This suggests that the pretrained feature space was doing most of the heavy lifting, while the logistic regression

classifier gave us a stable way to separate the 100 classes without adding too many trainable parameters.

At the same time, the approach is not perfect. Since the final model uses frozen embeddings, it does not fully adapt the backbone features to the specific classes in this dataset. A carefully regularized fine-tuning step might improve performance, but it would also introduce more opportunities to overfit and make the result harder to reproduce. If we continued working on this, the next improvements we would try are fold-based ensembling, more systematic tuning of the feature weights, and possibly light test-time augmentation, while still keeping the pipeline simple enough that another person could realistically rerun it.

7 Reproducibility

- Random seed: 144 for Python, NumPy, and PyTorch.
- Determinism: cuDNN benchmarking disabled and deterministic mode enabled in the project utility function.
- Environment: Python ≥ 3.10 with torch, torchvision, timm, numpy, pandas, Pillow, scikit-learn, and tqdm.
- Pretrained weights: downloaded automatically through timm/Hugging Face/Torch cache; no large weights are committed to git.

Exact reproduction command:

```
./scripts/run_best_85_submission.sh /path/to/ucsc-cse-144-spring-2026-final-project
```

Equivalent core command:

```
python -m cse144_project.make_embedding_submission \
  --data-dir /path/to/ucsc-cse-144-spring-2026-final-project --all-test \
  --models convnext_base.fb_in22k_ft_in1k_384 \
    swin_base_patch4_window12_384.ms_in22k_ft_in1k \
    vit_base_patch14_reg4_dinov2.lvd142m \
  --feature-block-weights 0.25,0.75,1.5 --blend-weights 1,0,0 \
  --c-grid 200 --knn-grid 5 --proto-temperature 6 --no-tta-flip \
  --batch-size 16 --workers 2 --seed 144 \
  --output submissions/SamarthRameshVirakChumKaggleSubmission_1036.csv
```

8 Team Contributions

- Solara Chum: worked on the embedding pipeline, helped compare pretrained model choices, checked validation results, and helped write and revise the final report.

- Samarth Ramesh : worked on the embedding pipeline, helped tune the model ensemble and classifier settings, generated the final Kaggle submission, and helped write and revise the final report.

Overall, we split the work evenly and both contributed to the modeling decisions, pipeline debugging, and final writeup.

9 References

- UCSC CSE 144 Spring 2026 Final Project handout and Kaggle competition page.
- PyTorch and TorchVision documentation for model training and inference workflows.
- timm model library documentation and pretrained model registry.
- scikit-learn documentation for LogisticRegression, StratifiedKFold, and KNeighborsClassifier.
- DINOv2, ConvNeXt, and Swin Transformer pretrained model sources as exposed through timm.

Link to trained model weights:

https://drive.google.com/file/d/133tf_q4_CNBO2Cavv2JUptZe7YdX8N0l/view?usp=sharing